МИНИСТЕРСТВО ЦИФРОВЫХ ТЕХНОЛОГИЙ РЕСПУБЛИКИ УЗБЕКИСТАН
ТАШКЕНТСКИЙ УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
имени Мухаммада ал-Хоразмий

Дисциплина: «Введение в программную инженерию»

ПРАКТИЧЕСКАЯ РАБОТА № 8

Паттерны проектирования

Вариант 19: разработка АРМ администратора салона красоты

Выполнил: **Тожикулов Асилбек**
Группа: **M320-24**
Вариант: **19**
Принял: _____

Ташкент, 2026

1. Тема практической работы

Паттерны проектирования.

Цель работы: закрепление знаний по теоретическим сведениям. Научиться выявлять в проекте типовые проблемы проектирования, подбирать для них архитектурный паттерн и паттерны GoF, обосновывать выбор по последствиям и применять паттерн в коде.

Вариант 19: разработка АРМ администратора салона красоты (BeautyDesk). Работа выполнена на основе требований и вариантов использования (ПР 5), модели проектирования (ПР 6) и интерфейса (ПР 7).

2. Обоснование выбора архитектурного паттерна

Для системы выбран архитектурный паттерн **«Многоуровневая архитектура» (Layered Architecture)** в варианте модульного монолита, намеченный в ПР 6. Система разделена на четыре слоя, каждый из которых обращается только к нижележащему:

1. **представление** – SPA в браузере (экраны S1–S7 из ПР 7) и REST-контроллеры (BookingController и др.);
2. **прикладной слой** – сервисы сценариев: BookingService, PaymentProcessor, ReportService;
3. **слой предметной области** – сущности Appointment, Client, Master, правила записи и статусов;
4. **слой инфраструктуры** – репозитории PostgreSQL, Redis, адаптеры Payme, Click, SMS, Telegram.

Таблица 1 – Обоснование выбора архитектурного паттерна

Критерий	Многоуровневая архитектура	Альтернатива: MVC без явного слоя сервисов
Целостность записи (FR-05)	правила записи сосредоточены в прикладном и доменном слоях, проверка и сохранение – в одной транзакции	логика расползается по контроллерам, риск двойных записей
Сопровождаемость (3 инкремента)	новый модуль = новые классы в своих слоях, остальные не меняются	контроллеры быстро разрастаются
Тестируемость (NFR-06)	сервисы тестируются без БД через интерфейсы репозиториев	сложно тестировать без веб-сервера
Замена внешних сервисов	адаптеры в инфраструктурном слое	вызовы API разбросаны по коду

Критерий	Многоуровневая архитектура	Альтернатива: MVC без явного слоя сервисов
Издержки	больше классов и «проходных» вызовов между слоями	меньше кода на старте

Многоуровневая архитектура выбрана, потому что ключевые требования проекта – надёжная запись без накладок, поэтапное развитие и автотесты – требуют отделить бизнес-правила от интерфейса и от базы данных. Издержка в виде дополнительных классов для команды из трёх разработчиков приемлема.

3. Проектные проблемы

Анализ результатов ПР 5–7 выявил три проблемы, которые повторяются в разных частях системы. Они описаны по форме «контекст – проблема – силы».

Таблица 2 – Проблема 1

Проблема 1	Разные способы оплаты визита
Контекст	UC-04 «Провести оплату визита», FR-08: администратор принимает оплату наличными, картой UzCard/Humo, через Payme и Click; на экране S4 (ПР 7) способ выбирается одной кнопкой
Проблема	у каждого способа свой протокол (касса смены, терминал, Merchant API Payme, API Click) и свой возврат; если писать всё в одном методе через «if способ == …», он будет расти с каждым новым способом и ломаться при изменениях
Силы	способ выбирается во время работы, а не при сборке программы; в Узбекистане появляются новые платёжные сервисы (Uzum Bank, Humo Pay) – добавление должно быть простым; каждый способ нужно тестировать отдельно; логика расчёта суммы и печати чека одинакова для всех способов

Таблица 3 – Проблема 2

Проблема 2	Множество реакций на изменение записи
Контекст	UC-01 и UC-03: после создания, переноса или отмены записи нужно отправить клиенту SMS/Telegram (FR-09), обновить кэш календаря (NFR-01) и статистику загрузки мастеров (UC-12); в ПР 6 «Журнал записи» получил самое высокое исходящее сцепление (Ce = 5)
Проблема	BookingService напрямую вызывает уведомления, кэш и статистику; каждое новое действие (например, начисление бонусов) требует менять сервис записи
Силы	запись – самая критичная операция и должна оставаться простой и надёжной; число реакций будет расти от инкремента к инкременту; сбой отправки SMS не должен мешать сохранению записи; реакции не зависят друг от друга

Таблица 4 – Проблема 3

Проблема 3	Сложный жизненный цикл записи
Контекст	FR-07 (ПР 5) задаёт 6 статусов записи: запланирована, подтверждена, в работе, завершена, отменена клиентом, неявка; в UC-03 и UC-04 действия разрешены не во всех статусах
Проблема	в каждом методе записи (confirm, pay, cancel, перенос) приходится проверять текущий статус длинными условиями; легко забыть проверку и, например, оплатить отменённую запись или отменить уже оплаченную
Силы	правила переходов должны быть в одном месте и легко читаться; при добавлении статуса (например, «опоздание») не должны меняться все методы; ошибка перехода должна давать понятное сообщение администратору (эвристика Нильсена 9 из ПР 7)

4. Таблица «проблема – паттерн – последствия»

Таблица 5 – Проблема – паттерн – последствия

Проблема	Паттерн GoF	Выгоды	Издержки
1. Разные способы оплаты	Стратегия (Strategy), поведенческий	каждый способ оплаты – отдельный класс; новый способ добавляется без изменения кода кассы (принцип открытости/закрытости); способы тестируются по отдельности; способ выбирается во время работы	больше классов; клиентский код должен знать, какую стратегию выбрать (решается простой фабрикой по кнопке экрана)
2. Множество реакций на изменение записи	Наблюдатель (Observer), поведенческий	BookingService не знает о подписчиках – сцепление «Журнала записи» снижается (Ce 5 → 4, рефакторинг из ПР 6); новая реакция = новый подписчик; сбой одного подписчика можно изолировать	порядок вызова подписчиков не гарантирован; сложнее отлаживать цепочку событий; нужно не забывать отписывать объекты
3. Сложный жизненный цикл записи	Состояние (State), поведенческий	правила каждого статуса собраны в своём классе; нет длинных if/switch; запрещённое действие даёт понятную ошибку; новый статус добавляется новым классом	6 дополнительных классов; при небольшом числе статусов паттерн может быть избыточен; переходы распределены по классам, а не видны в одной таблице

5. Диаграммы классов применения паттернов

На диаграммах участники названы терминами предметной области салона, а их роль в паттерне указана тёмной меткой над классом (Context, Strategy, Observer и т. д.).

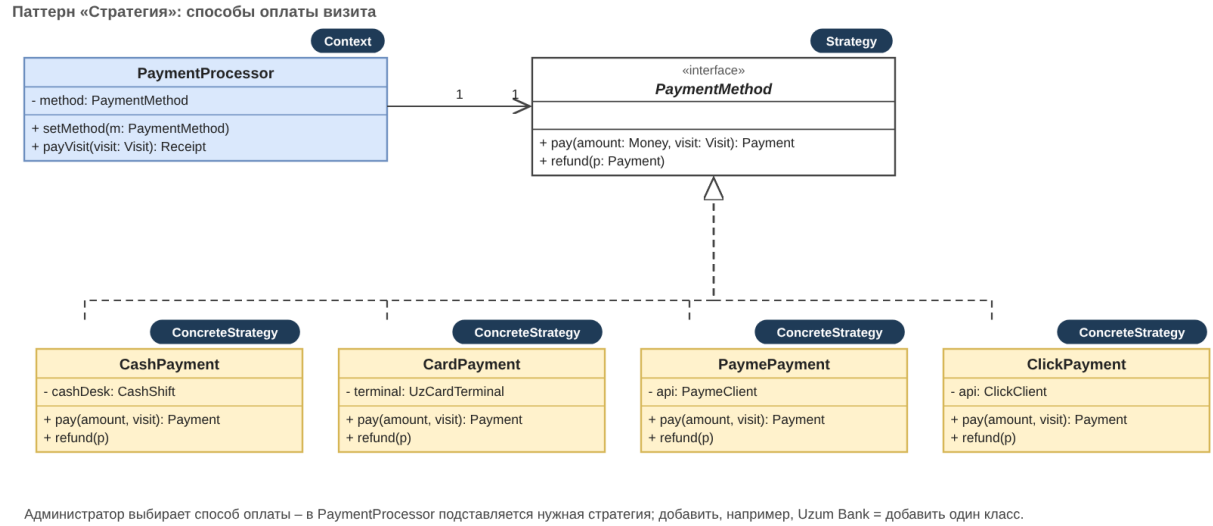


Рисунок 1 – Применение паттерна «Стратегия» к оплате визита

Роли: PaymentProcessor (касса) – Context, хранит выбранный способ оплаты и проводит оплату визита; PaymentMethod – Strategy, общий интерфейс; CashPayment, CardPayment, PaymePayment, ClickPayment – ConcreteStrategy, каждая работает со своим внешним сервисом.

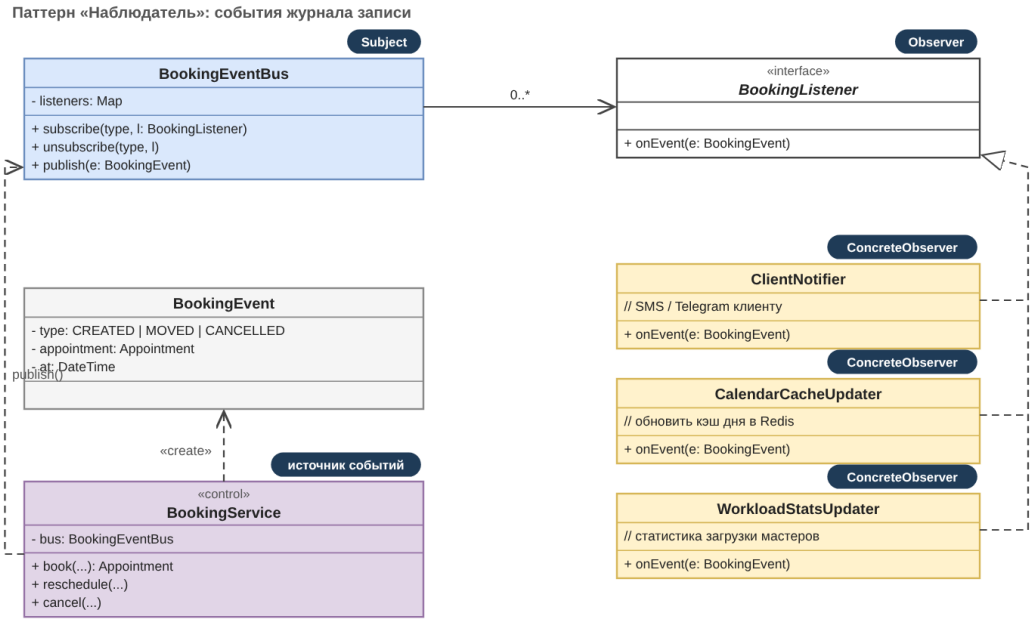


Рисунок 2 – Применение паттерна «Наблюдатель» к событиям журнала записи

Роли: BookingEventBus (шина событий записи) – Subject, хранит подписчиков по типу события; BookingListener – Observer; ClientNotifier, CalendarCacheUpdater,

WorkloadStatsUpdater – ConcreteObserver. BookingService только публикует событие BookingEvent.

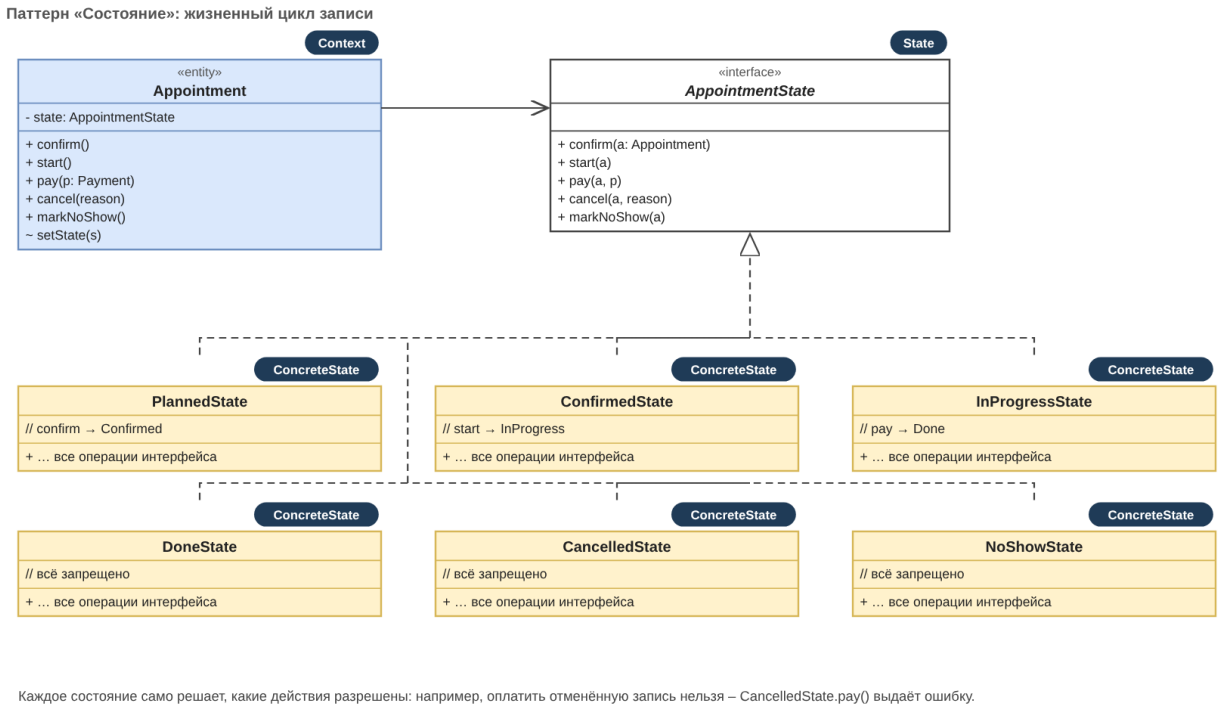


Рисунок 3 – Применение паттерна «Состояние» к жизненному циклу записи

Роли: Appointment (запись) – Context, делегирует действия текущему состоянию; AppointmentState – State; шесть классов статусов из FR-07 – ConcreteState. Например, ConfirmedState.start() переводит запись в InProgressState, а CancelledState.pay() выдаёт ошибку «Нельзя оплатить отменённую запись».

6. Фрагмент кода: паттерн «Наблюдатель»

Для реализации выбран паттерн «Наблюдатель», так как он одновременно решает проблему 2 и выполняет рефакторинг из ПР 6. Код написан на Python 3 (78 строк, рисунки 4–5).

booking_events.py — Visual Studio Code

```python
"""BeautyDesk: паттерн «Наблюдатель» для событий журнала записи."""
from abc import ABC, abstractmethod
from collections import defaultdict
from dataclasses import dataclass
from datetime import datetime


@dataclass
class BookingEvent:                          # что произошло с записью
    type: str                                # CREATED | MOVED | CANCELLED
    client: str
    master: str
    start: datetime


class BookingListener(ABC):                  # Observer
    @abstractmethod
    def on_event(self, e: BookingEvent): ...


class BookingEventBus:                       # Subject
    def __init__(self):
        self._listeners = defaultdict(list)

    def subscribe(self, event_type, listener):
        self._listeners[event_type].append(listener)

    def publish(self, e: BookingEvent):
        for listener in self._listeners[e.type]:
            listener.on_event(e)

```

Рисунок 4 – Код: событие, интерфейс наблюдателя и шина событий (Subject)

booking_events.py — Visual Studio Code

```python
class ClientNotifier(BookingListener):       # ConcreteObserver 1
    def on_event(self, e):
        text = {"CREATED": "Вы записаны", "MOVED": "Запись перенесена",
                "CANCELLED": "Запись отменена"}[e.type]
        print(f"  [SMS] {e.client}: {text} — {e.start:%d.%m %H:%M}, мастер {e.master}")


class CalendarCacheUpdater(BookingListener): # ConcreteObserver 2
    def on_event(self, e):
        print(f"  [Кэш] журнал на {e.start:%d.%m} обновлён")


class WorkloadStats(BookingListener):        # ConcreteObserver 3
    def __init__(self):
        self.booked = defaultdict(int)

    def on_event(self, e):
        self.booked[e.master] += 1 if e.type == "CREATED" else -1 if e.type == "CANCELLED" else 0
        print(f"  [Статистика] записей у {e.master}: {self.booked[e.master]}")


class BookingService:                        # источник событий — ничего не знает о подписчиках
    def __init__(self, bus: BookingEventBus):
        self.bus = bus

    def book(self, client, master, start):
        print(f"Запись: {client} → {master}")
        self.bus.publish(BookingEvent("CREATED", client, master, start))

    def cancel(self, client, master, start):
        print(f"Отмена: {client}")
        self.bus.publish(BookingEvent("CANCELLED", client, master, start))


if __name__ == "__main__":
    bus, stats = BookingEventBus(), WorkloadStats()
    for t in ("CREATED", "MOVED", "CANCELLED"):
        bus.subscribe(t, ClientNotifier())
        bus.subscribe(t, CalendarCacheUpdater())
    bus.subscribe("CREATED", stats)
    bus.subscribe("CANCELLED", stats)

    service = BookingService(bus)
    service.book("Азиза Каримова", "Дилноза", datetime(2026, 12, 26, 14, 0))
    service.book("Ирода Алиева", "Дилноза", datetime(2026, 12, 26, 16, 0))
    service.cancel("Ирода Алиева", "Дилноза", datetime(2026, 12, 26, 16, 0))
```

Рисунок 5 – Код: конкретные наблюдатели, сервис записи и пример использования

Пояснения к коду:

– BookingEvent – данные события: тип, клиент, мастер и время;

– BookingListener – абстрактный класс с методом on_event (роль Observer);

– BookingEventBus хранит словарь «тип события → список подписчиков»; метод publish вызывает on_event у всех подписчиков нужного типа (Subject);

– ClientNotifier, CalendarCacheUpdater и WorkloadStats – три независимые реакции (ConcreteObserver); статистика подписана только на создание и отмену;

– BookingService в методах book и cancel только публикует событие и ничего не знает о том, кто на него реагирует.

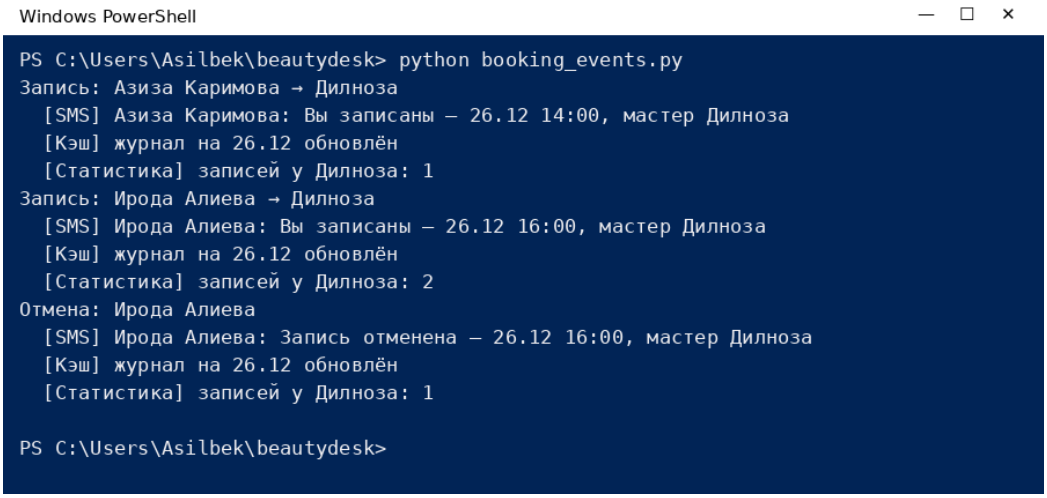


Рисунок 6 – Результат выполнения программы

Результат. При каждой записи все три подписчика срабатывают автоматически: клиенту уходит SMS, кэш журнала обновляется, счётчик записей мастера растёт (1, затем 2). При отмене клиент получает сообщение об отмене, а статистика уменьшается до 1. Чтобы добавить, например, начисление бонусов, достаточно написать класс BonusListener и подписать его – BookingService не меняется.

7. Возможный антипаттерн и способ исправления

Без выбранных паттернов в проекте почти неизбежно возник бы антипаттерн **«Божественный объект» (God Object)**: класс BookingService, который сам проверяет статусы, рассчитывает сумму, проводит оплату через все платёжные системы, отправляет SMS и Telegram, обновляет кэш и считает статистику.

Таблица 6 – Антипаттерн «Божественный объект»

Аспект	Описание
Симптомы	класс на 1000+ строк и десятки методов; длинные цепочки if/elif по способу оплаты и статусу записи; почти любое изменение (новый способ оплаты, новый статус, новое уведомление) требует правки этого класса; конфликты слияния у всех трёх разработчиков; ошибка в отправке SMS ломает сохранение записи; класс невозможно протестировать без БД, Redis и внешних API
Причина	логика добавляется «туда, где уже есть похожий код», особенно при спешке перед сдачей инкремента
Последствия	высокое сцепление и низкая связность (как показала оценка в ПР 6), рост числа ошибок, замедление разработки к инкременту 3
Способ исправления	выделить обязанности в отдельные классы: оплату – в стратегии PaymentMethod, реакции на события – в наблюдателей, правила статусов – в классы состояний; BookingService оставить координатором сценария; закрепить правило ревью кода: «класс сервиса – не более 200 строк, без switch по типу»

Неуместное применение паттернов тоже опасно: например, вводить «Наблюдатель» для действий, которые обязательно должны выполниться в одной транзакции с записью (блокировка слота), нельзя – порядок и надёжность подписчиков не гарантированы. Поэтому блокировка слота оставлена прямым вызовом ISlotLock, а через события идут только второстепенные реакции.

8. Вывод о проделанной работе

В работе для системы «АРМ администратора салона красоты» обоснован архитектурный паттерн «Многоуровневая архитектура», который отделяет бизнес-правила записи от интерфейса и инфраструктуры и соответствует требованиям надёжности, тестируемости и поэтапного развития.

По результатам ПР 5–7 выявлены три проектные проблемы: разнородные способы оплаты, множество реакций на изменение записи и сложный жизненный цикл записи с шестью статусами. Для них выбраны паттерны GoF «Стратегия», «Наблюдатель» и «Состояние»; выгоды и издержки каждого решения сведены в таблицу. Построены три диаграммы классов, на которых участники названы терминами предметной области, а их роли в паттернах обозначены метками.

Паттерн «Наблюдатель» реализован на Python (78 строк): сервис записи публикует события, а уведомление клиента, обновление кэша и статистика реагируют на них независимо, что снижает сцепление модуля «Журнал записи». Описан антипаттерн «Божественный объект», который возник бы без паттернов, его симптомы и способ исправления, а также пример неуместного применения паттерна.